

Finale file formats

A reader's specification for `.mus` and `.musx`

Reconstructed by the finale-file-parser project. All hex dumps are synthetic.

Contents

1. [Scope and provenance](#)
2. [Which formats are supported](#)
3. [Byte order, compression, and encryption](#)
4. [The `.mus` file header](#)
5. [Container: how the payload is found](#)
6. [Records: framing and addressing](#)
7. [The entry pool](#)
8. [The `.musx` container](#)
9. [Record catalogue](#)

1. Scope and provenance

This document describes the binary layout of Finale's `.mus` and `.musx` files as reconstructed by the `finale-file-parser` project. Finale was discontinued in 2024 and its format was never published; everything here comes from reverse engineering against a curated corpus, from two vendored Coda documents, and from prior community research.

It is written for someone implementing a reader. Every structure is given as a C-style declaration, a field table, and a hex dump, in the manner of the Aseprite file specification.

***Confidence.** Each structure below is annotated with what it was verified against. Where this project's findings contradict Coda's own documentation, that is stated explicitly rather than quietly resolved — there is one such case, the note alteration encoding in §7.2.*

NO CORPUS BYTES APPEAR IN THIS DOCUMENT

Every hex dump is **synthetic**, constructed by the generator that produced this PDF. The test corpus is licensed third-party music and its bytes are not reproducible here. Synthetic data also lets each dump isolate the field under discussion instead of burying it in a real record.

This document is generated from the parser. Offsets, sizes and constants are imported from the reading code rather than retyped, so a layout here cannot silently drift from the implementation that reads it.

2. Which formats are supported

Three containers, spanning roughly two decades of Finale releases.

CONTAINER	FINALE ERA	PAYLOAD CODEC	BYTE ORDER	RECORD ENCODING
.mus (DCL)	2001–2005	PKWARE DCL (“implode”)	writing platform's	fixed 16-byte ETF rows
.mus (2011)	2011–2012	zlib (chained streams)	little-endian	self-identifying variable-length
.musx	2012–2024	ZIP + XOR cipher, then zlib	little-endian	EnigmaXML elements

CORPUS COVERAGE

The figures the parser is measured against, over 639 documents:

- .musx — 401 of 401 build a score
- 2011-era .mus — 99 of 99
- DCL-era .mus — 131 of 139

The eight that do not are the files rather than the reader: six are blank scores refused deliberately, one is a mirror, one an incomplete export.

Not covered. Finale versions between 2005 and 2011, and before 2001, are absent from the corpus and therefore absent from this document. Nothing here should be assumed to hold for them.

3. Byte order, compression, and encryption

Byte order is the writing machine's

For DCL-era .mus files, integers are written in the byte order of the machine that saved the file: **big-endian on Mac, little-endian on Windows**. This governs the pool records and every field inside them.

It is *detected, not assumed*. The first pool record's kind is always 15, a value that reads as 15 only one way round. Over the corpus: 102 little-endian, 37 big-endian, all 139 walking to the last byte exactly.

The 2011 era is **all little-endian** even though those files are Mac-written — the Intel transition had happened. Do not infer byte order from platform for that era.

Compression

DCL era (2001–2005)

A chain of PKWARE DCL (“implode”) records beginning at `0x200`. No Python standard-library module reads this; a decoder is required. Measured inflation: $3.25\times$ – $4.51\times$.

2011 era

A chain of raw zlib streams, the first located by scanning for `78 9c`. Measured inflation: $5.87\times$ – $8.63\times$.

.musx

A ZIP archive. The member `score.dat` is XOR-encrypted, and decrypts to zlib-compressed EnigmaXML.

Decompression bombs. Any reader must cap total inflated output. This implementation refuses past 64 MiB across the whole chain, which leaves roughly $90\times$ headroom over the largest real corpus payload (699,585 bytes).

The score.dat cipher

score.dat is XOR-encrypted with a keystream from a BSD `rand()` linear congruential generator seeded with a fixed constant. The generator is **reseeded at every 128 KiB boundary**, so the keystream is one constant block repeated end to end.

These cipher parameters were not discovered by this project. They come from [denigma](#) (MIT), whose source credits [Deguerre](#).

4. The .mus file header

Both .mus eras share a header. Its first 0xA0 bytes carry the version banner and two provenance stamps.

A synthetic .mus header. The banner is the primary version evidence; the two provenance stamps say which application and platform wrote the file.

```
// byte order: little-endian (this example)
struct MusFileHeader {
    char[64]  banner;                // +0x20  NUL-terminated; tail of a previ...
    uint32    created.date;          // +0x66  creation date stamp
    char[4]    created.app;          // +0x70  application tag, e.g. FIN
    char[4]    created.platform;     // +0x74  MAC or WIN
    uint32    modified.date;         // +0x8c  last-modified date stamp
    char[4]    modified.app;         // +0x96  application tag
    char[4]    modified.platform;    // +0x9a  MAC or WIN
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x20–0x5f	64	char[64]	banner	NUL-terminated; tail of a previous longer banner may follow
	0x66–0x69	4	uint32	created.date	creation date stamp
	0x70–0x73	4	char[4]	created.app	application tag, e.g. FIN
	0x74–0x77	4	char[4]	created.platform	MAC or WIN
	0x8c–0x8f	4	uint32	modified.date	last-modified date stamp
	0x96–0x99	4	char[4]	modified.app	application tag
	0x9a–0x9d	4	char[4]	modified.platform	MAC or WIN

Example bytes (160 shown), tinted to match the table above.

```
0000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0020  46 69 6e 61 6c 65 28 52 29 20 32 30 30 35 20 66  Finale(R) 2005 f
0030  6f 72 20 57 69 6e 64 6f 77 73 00 00 00 00 00 00  or Windows.....
0040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0060  00 00 00 00 00 00 80 3a 09 00 00 00 00 00 00 00  .....
0070  46 49 4e 00 57 49 4e 00 00 00 00 00 00 00 00 00  FIN.WIN.....
0080  00 00 00 00 00 00 00 00 00 00 00 00 10 3b 09 00  .....;..
0090  00 00 00 00 00 00 46 49 4e 00 57 49 4e 00 00 00  .....FIN.WIN...
```

The banner field is fixed-size and is *not* zero-filled when Finale rewrites it, so a shorter banner can leave the tail of a previous, longer one behind. Everything from the first NUL onward must be discarded.

A stamp is all-or-nothing: an implausible date or an empty application tag means the whole stamp is untrustworthy, because a reader cannot tell which half of a partial stamp to believe.

The banner is the primary version evidence. A regular expression of the form `Finale\\(R\\)\\s+(\\d{4})` against the banner text yields the year. An unrecognised banner should leave the year unset with the raw text preserved, rather than failing — an unknown variant stays inspectable that way.

5. Container: how the payload is found

A .mus payload is **not one blob**. It is a handful of compressed *pools*, laid end to end.

POOL	KIND	HOLDS
others	15	most record types, keyed by one cmper
details	16	records keyed by a pair of cmpers
entries	17	the notes themselves
text	18	human-readable strings

DCL era: a labelled chain

Records run from `0x200` to the last byte of the file, with no gaps. The DCL era **labels** its pools; the zlib era does not.

A DCL-era pool record, little-endian (Windows-written). Records lie end to end from 0x200 to the last byte of the file, with no gaps.

```
// byte order: little-endian
struct DclPoolRecord {
    uint16    kind;                // +0x0    15 others, 16 details, 17 entri...
    uint32    length;              // +0x2    whole record, this 10-byte head...
    uint32    checksum;            // +0x6    omitted entirely when length == 6
    uint8[]   stream;              // +0xa    PKWARE DCL data, length - 10 bytes
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	kind	15 others, 16 details, 17 entries, 18 text
	0x02–0x05	4	uint32	length	whole record, this 10-byte header included
	0x06–0x09	4	uint32	checksum	omitted entirely when length == 6
	0x0a–0x10	7	uint8[]	stream	PKWARE DCL data, length - 10 bytes

Example bytes (17 shown), tinted to match the table above.

```
0000  0f 00 11 00 00 00 9c 21 44 00 00 04 0e c5 b3 9a  ....!D.....
0010  2f                                     /
```

length counts its own header. The chain is walked by adding length to the current position. This is also why a fixed 0x20A works as “where the first stream starts”: 0x200 plus the ten-byte header.

A length of exactly 6 means the pool is **empty** — kind and length and nothing else, no checksum and no stream.

The identical record written big-endian (Mac). Byte order is detected from this first record rather than assumed: its kind is always 15, which reads as 15 only one way round.

```
// byte order: big-endian
struct DclPoolRecord {
    uint16    kind;                // +0x0    15 – only 15 one way round
    uint32    length;              // +0x2
    uint32    checksum;            // +0x6
    uint8[]   stream;              // +0xa
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	kind	15 — only 15 one way round
	0x02–0x05	4	uint32	length	
	0x06–0x09	4	uint32	checksum	
	0x0a–0x10	7	uint8[]	stream	

Example bytes (17 shown), tinted to match the table above.

```
0000  00 0f 00 00 00 11 00 44 21 9c 00 04 0e c5 b3 9a  ....D!.....
0010  2f                                     /
```

2011 era: an unlabelled chain

The first zlib stream is found by scanning for the 78 9c header. Streams follow one another; the pools are identified by *order*, not by a label, which is why a reader must know the sequence rather than read it.

6. Records: framing and addressing

What a record is

A record is **a row in a table, not an object in a tree**. Almost every wrong intuition about these formats comes from expecting a document tree, so it is worth stating plainly before any bytes.

A record is three things:

A tag

What kind of thing this is — measSpec, gfhold. Think table name.

One or more keys

What it is *about*. A measSpec keyed 7 is measure 7; a gfhold keyed (3, 7) is staff 3, measure 7. Think primary key.

A payload

Bytes whose meaning depends entirely on the tag. Think columns. There is no self-description inside a payload: without the tag, the bytes mean nothing.

Three consequences matter to anyone writing a reader.

There are no pointers. Nothing holds a file offset. A `gfhold` does not point at a `frameSpec`; it contains the number 41, and somewhere there is a `frameSpec` whose key is 41. Resolution is a lookup by key equality — a join, not a dereference. Even the entry pool's `next` and `prev`, which look like a linked list, are entry *numbers*. The physical order of every record in a file could be shuffled without losing anything.

Records do not nest. Composition happens by reference. The music of a measure is not inside the measure's record; it is reached by a chain of key lookups.

A logical record is not always a physical one. In the 2011 era they are one to one — each record carries its own length. In the DCL era a record is a fixed 16-byte row, and anything larger continues into further rows under the same tag and key. ETF calls each row an *incidence*. So a record is the concatenation of its rows, and fields are addressed by offset into that concatenation, not into any one row.

THE HIERARCHY, AND WHERE CONTAINMENT ACTUALLY HAPPENS

“A pool contains records, and records contain entries” is half right. Pools do contain records. But an entry **is** a record: it lives in its own pool, keyed by entry number, a peer of `measSpec` rather than a child of anything.

What misleads is `frameSpec`, which looks like containment and is not. It holds `startEntry` and `endEntry` — two integers naming a range of keys. Delete the `frameSpec` and its entries are still there, orphaned but intact.

```
file
├─ pools
│   └─ records
│       └─ (entries pool)
│           └─ entry
│               └─ notes
├─ others
├─ details
├─ entries
└─ text
    a pool is a flat run of them
    a record: one chord or rest at one point in time
    6-byte note records – real containment
```

Notes are the one genuine nesting in the format. An entry's payload holds its own `noteCount` and the note records themselves; they have no independent key and cannot be addressed from outside. Everything else is reference by key.

In one line: **pools contain records; records reference each other by key; only entries contain anything, and what they contain is notes.**

2011 era: self-identifying records

A 2011-era record: tag 176 (`measSpec`), key 1, score part. One record occupies $10 + \text{length} + 4$ bytes.

```
// byte order: little-endian
struct MusRecord2011 {
    uint16    tag;                // +0x0    record type; .musx names the sa...
    uint16    cmper;             // +0x2    the (n) in an ETF ^XX(n) – the...
    uint16    part;              // +0x4    0 for the score, then 1, 2, .....
    uint32    length;            // +0x6    payload size in bytes
    uint8[]   payload;           // +0xa    length bytes
    uint32    trailer;           // +0x16   the length again
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	tag	record type; .musx names the same types
	0x02–0x03	2	uint16	cmper	the (n) in an ETF ^XX(n) — the record's key
	0x04–0x05	2	uint16	part	0 for the score, then 1, 2, ... per linked part
	0x06–0x09	4	uint32	length	payload size in bytes
	0x0a–0x15	12	uint8[]	payload	length bytes
	0x16–0x19	4	uint32	trailer	the length again

Example bytes (26 shown), tinted to match the table above.

```
0000  b0 00 01 00 00 00 0c 00 00 00 60 00 58 02 04 00  .....X...
0010  01 00 00 00 00 00 0c 00 00 00  .....X...
```

These records are **self-identifying**: each carries its own key, so nothing outside the record is needed to address it — no directory, no key array, no positional convention.

Records of one tag sit together in a section; sections may be separated by two-byte zero padding, which a walk must skip.

Why this took so long to find: earlier attempts anchored on a payload field located by searching for a value already known from a paired .musx, then read *forward*. The key sits ten bytes *behind* that anchor, so it was never in view.

DCL era: fixed ETF rows

The same information, encoded completely differently. Where a 2011 record carries its own length, a DCL row is always 16 bytes and a long record simply continues into the next row.

A 2001–2005 others row: fixed 16 bytes, ETF's two-character tag.

```
// byte order: writing platform's
struct Dcl0thersRow {
    uint16    cmper;             // +0x0    the (n) in an ETF ^XX(n)
    uint16    tag;              // +0x2    two characters stored as a u16
    uint8[12] data;             // +0x4    ETF's 6 twobyte values (or 3 fo...
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
 0x00–0x01	2	uint16	cmper	the (n) in an ETF ^XX(n)
 0x02–0x03	2	uint16	tag	two characters stored as a u16
 0x04–0x0f	12	uint8[12]	data	ETF's 6 twobyte values (or 3 fourbytes)

Example bytes (16 shown), tinted to match the table above.

0000 01 00 53 4d 60 00 58 02 04 00 01 00 00 00 00 00 ..SM`.X.....

The tag is a u16, not two bytes. On a little-endian file its characters come out reversed: ^MS is written SM, ^&a is written a&. Reading the pair verbatim finds no known tag in 102 of 139 corpus documents, which is how this was noticed.

A record too big for one row **runs on into further rows** under the same tag and key. ETF calls each row an *incidence*. A record's payload is its rows' data concatenated in file order.

A details row. Two keys rather than one — details are addressed by a pair, which is how a record hangs off a (staff, measure) intersection.

```
// byte order: writing platform's
struct DclDetailsRow {
    uint16    cmper1;           // +0x0    first key
    uint16    cmper2;           // +0x2    second key
    uint16    tag;              // +0x4    two characters as a u16
    uint8[10] data;             // +0x6    ETF's 5 twobytes
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
 0x00–0x01	2	uint16	cmper1	first key
 0x02–0x03	2	uint16	cmper2	second key
 0x04–0x05	2	uint16	tag	two characters as a u16
 0x06–0x0f	10	uint8[10]	data	ETF's 5 twobytes

Example bytes (16 shown), tinted to match the table above.

0000 01 00 03 00 46 47 01 00 00 00 00 00 00 00 00 00FG.....

How items point to each other

There is no pointer arithmetic anywhere in these formats. Addressing is by **key**, and the keys are small integers.

cmper (“comparator”)

A record's key — the (n) in ETF's ^XX(n) notation. For a measSpec it is the measure number; for a staffSpec, the staff.

cmper1, cmper2

Details records are keyed by a *pair*, which is how a record hangs off an intersection — a gfhold, for instance, at (staff, measure).

inci (“incidence”)

When several records share a key, the incidence distinguishes them. In the DCL era it is also how one logical record spills across several 16-byte rows.

part

0 for the score; 1, 2, ... for each linked part. A part's record overrides the score's for that part only.

FINDING THE MUSIC: THE FRAME CHAIN

This is the central traversal, and it is worth stating as a sequence:

1. A `gfhold` (“frame hold”) is keyed by (staff, measure) and names up to four **frames**, one per layer.
2. Each frame id keys a `frameSpec`, which gives a `startEntry` and an `endEntry`.
3. Those are entry numbers into the entry pool. The entries between them, inclusive, are that layer's music for that measure.
4. Each entry carries its own `next` pointer, so the run can also be walked as a linked list.

An entry reached by more than one frame is a corrupt document, not a shared voice. A reader should refuse it rather than emit the notes twice.

Known record tags

One vocabulary in three spellings: ETF's two-character tags, the 2011 era's numbers, and EnigmaXML's symbolic names. The three tiers below are kept apart on purpose — a payload-confirmed identification and a key-sequence guess are not the same claim.

TIER A — DECODED AND PAYLOAD-CONFIRMED

Parsed by this project, with fields verified against paired `.musx` documents.

TAG	ETF	NAME	POOL	DESCRIPTION
109		<code>clefOptions</code>	others	Clef definition table, read as a strided array
121		<code>articDef</code>	others	An articulation's definition; <code>charMain</code> is the glyph
146	<code>^FR</code>	<code>frameSpec</code>	others	Entry range for one layer of one measure
159		<code>instUsed</code>	others	The staves the score lays out, in layout order
176	<code>^MS</code>	<code>measSpec</code>	others	Per measure: width, key, beats, divbeat, barline
203		<code>repeatBack</code>	others	Backward repeat barline, keyed by measure
204		<code>repeatEndingStart</code>	others	Ending bracket, keyed by measure
206		<code>repeatPassList</code>	others	Which passes an ending is taken on
231	<code>^IS</code>	<code>staffSpec</code>	others	Per staff; transposition at <code>+0x14</code>
1009		<code>articAssign</code>	details	The articulation on an entry; names an <code>articDef</code>
1044	<code>^GF</code>	<code>gfhold</code>	details	(staff, measure) to clef and up to four frames
1057		<code>staffGroup</code>	details	Brace or bracket over a run of staves
1072		<code>tupletDef</code>	details	Keyed by entry, not by (staff, measure)
1108		<code>lyrDataVerse</code>	details	Verse lyrics

TAG	ETF	NAME	POOL	DESCRIPTION
—	^eE	entry	entries	The notes; fixed 38-byte slots

TIER B — NAMED BY KEY-SEQUENCE MATCHING ONLY

Identified by matching each tag's (cmper, part) sequence against the .musx others pool. **Treat these as leads, not facts:** a tag whose record count collides with another's could be mismatched, and none has been confirmed by payload content. “Docs” is how many corpus documents agreed.

TAG	NAME	DOCS
124	channelPlayData	80
126	chordSuffixPlay	85
131	drumLibName	86
134	durAllot	91
136	execShape	85
140	fretboardSymbol	91
144	fontName	5
147	lockMeas	81
149	fretInst	90
163	layerAtts	85
165	metaArtic	80
168	metaDynam	81
169	metaKeySig	80
170	metaRepeat	81
171	metaShape	80
172	metaStaffStyle	80
173	metaTimeSig	81
235	shapeExprDef	10
242	textExpressionEnclosure	14
315	volumeValue	14

fontName (5 documents) and shapeExprDef (10) rest on so few agreements that they are better read as guesses.

TIER C — OBSERVED BUT UNIDENTIFIED

189 distinct tags appear in the 2011 others pool alone, of which the tiers above account for roughly thirty. The rest are read, keyed and carried, but their meaning is unknown. The most frequent, by record count across the corpus:

others

213 and 215 (25,576 each), 140 (18,624), 214 (12,681), 125 (12,612), 148 (11,887), 126 (11,208), 183 (10,114), 192 and 241 (7,353 each), 217 (6,638)

details

1043 (166,524), 1064 (8,162), 1063 (2,162), 1066 (1,483), 1034 (1,164), 1060 (1,156)

DCL (ETF)

IV (15,274), IK (15,024), SD (13,838), sL and sb (13,802 each), FB (8,448), IX (7,415), DT (4,429), TX (4,068); and in details DF (74,730), fb (63,324), DN (18,647)

Two hints from the counts. Several pairs share an exact record count — 213/215, 192/241, and sL/sb — which across a whole corpus usually means a definition-and-assignment couple like `articDef/articAssign`. And details tag 1043, at 166,524 records, is roughly sixteen times more common than `gfhold`, which suggests something per entry or per note rather than per measure.

***What this means for scope.** The tags decoded here are not the common ones; they are the ones on the path from file to notes. Everything above is largely layout, spacing, fonts, page format and text blocks. By tag count this specification covers under a fifth of the vocabulary, and by record count rather less — it is a specification for extracting the music, not a complete description of the file.*

7. The entry pool

The notes. This is the one structure both `.mus` eras share byte-for-byte, differing only in integer byte order.

An entry's first slot. The pool is a flat array of fixed 38-byte slots, each tagged with the entry it belongs to.

```
// byte order: little-endian
struct EntrySlotFirst {
    uint32  entnum;           // +0x0    the (n) in ETF ^eE(n)
    uint16  slot;             // +0x4    0 for an entry's first slot, th...
    uint32  prev;             // +0x6    previous entry number, 0 if none
    uint32  next;             // +0xa    next entry number, 0 if none
    uint16  dura;             // +0xe    written duration in EDU; 1024 =...
    int16   pos;              // +0x10   manual positioning in EVPU, signed
    uint32  flags;             // +0x12   SETBIT 0x80000000 always; NOTEB...
    uint16  extflags;         // +0x16   extended flags
    uint16  noteCount;        // +0x18   how many note records follow, a...
    NoteRec note[0];          // +0x1a   TCD (2) + note flag (4)
    NoteRec note[1];          // +0x20   the first slot holds two
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x03	4	uint32	entnum	the (n) in ETF ^eE(n)
	0x04–0x05	2	uint16	slot	0 for an entry's first slot, then 1, 2, ...
	0x06–0x09	4	uint32	prev	previous entry number, 0 if none
	0x0a–0x0d	4	uint32	next	next entry number, 0 if none
	0x0e–0x0f	2	uint16	dura	written duration in EDU; 1024 = quarter note
	0x10–0x11	2	int16	pos	manual positioning in EVPU, signed
	0x12–0x15	4	uint32	flags	SETBIT 0x80000000 always; NOTEBIT 0x40000000
	0x16–0x17	2	uint16	extflags	extended flags
	0x18–0x19	2	uint16	noteCount	how many note records follow, across slots
	0x1a–0x1f	6	NoteRec	note[0]	TCD (2) + note flag (4)
	0x20–0x25	6	NoteRec	note[1]	the first slot holds two

Example bytes (38 shown), tinted to match the table above.

```
0000 65 00 00 00 00 00 64 00 00 00 66 00 00 00 00 04 e....d...f....
0010 00 00 00 00 00 c0 00 00 02 00 c0 03 00 00 00 00 .....
0020 01 04 00 00 00 00 .....

```

Continuation slots (index > 0) carry only notes, 5 per slot, from offset 6. An entry may carry up to 12 notes.

Both eras share this layout. What differs is only which way round the integers are written. That the layout really is shared is the corpus's verdict, not an assumption: across 136 DCL-era documents the slots tile the pool exactly and every entry has SETBIT set.

The note record

A note record: six bytes, two of them carrying both pitch and alteration.

```
// byte order: little-endian
struct NoteRec {
    uint16  tcd;                // +0x0    upper 12 bits pitch (signed), l...
    uint32  flags;              // +0x2    TIE_START 0x40000000, TIE_END 0...
};

```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	tcd	upper 12 bits pitch (signed), low nibble alteration
	0x02–0x05	4	uint32	flags	TIE_START 0x40000000, TIE_END 0x20000000

Example bytes (6 shown), tinted to match the table above.

```
0000 09 04 00 00 00 40 .....@

```

The alteration is sign-and-magnitude, not two's complement. Bit 3 is the sign. `eeppd.txt` describes it as “a signed quantity ... -8 to +7”, which reads as two's complement; the corpus disagrees, and the corpus wins. Observed: 0x0 → 0, 0x1 → +1, 0x9 → −1.

DURATIONS

Durations are in **EDU**, where 1024 is a quarter note and 4096 a whole note. Dotted values are the plain value plus half again, so 1536 is a dotted quarter. Across 136 DCL-era corpus documents, 71,801 durations took just 16 distinct values — every one a note value with 0–2 dots, which is itself evidence the field was read correctly.

8. The .musx container

A **.musx** is a ZIP archive. Unlike the two **.mus** eras it needs no bespoke container walking — a standard ZIP reader opens it — but its payload member is encrypted.

MEMBER	CONTENTS
mimetype	the fixed string <code>application/vnd.makemusic.notation</code>
META-INF/container.xml	archive manifest
NotationMetadata.xml	document metadata
score.dat	the score — XOR-encrypted, then zlib
graphics/*, presets/*	embedded artwork and presets

Reading score.dat

1. Read the member's bytes from the archive.
2. XOR with the LCG keystream (§3), which resets every 128 KiB.
3. Inflate the result with zlib.
4. The output is **EnigmaXML**, an XML document whose elements carry the same record types the binary formats encode numerically.

Treat the archive as hostile. A ZIP may declare member sizes that do not match its content, repeat member names, or expand enormously. Cap the inflated size, reject duplicate names, and never resolve a member path outside the archive root.

Why one reader covers all three

EnigmaXML names its record types symbolically — `measSpec`, `gfhold`, `entry` — where the 2011 era numbers them and the DCL era uses ETF's two-character tags. These are three spellings of one vocabulary. A reader that converges all three onto a single document model gets the rest of the pipeline for free, which is what this project does: the container differs, the music does not.

CONCEPT	.MUSX	2011 .MUS	DCL .MUS
measure	<code>measSpec</code>	176	<code>^MS</code>
staff	<code>staffSpec</code>	231	<code>^IS</code>
frame	<code>frameSpec</code>	146	<code>^FR</code>
frame hold	<code>gfhold</code>	1044	<code>^GF</code>
note group	<code>entry</code>	entry pool	entry pool

9. Record catalogue

Every record type this project decodes, with the offsets its reader actually uses. Fields not listed are present in the payload but not yet established; they are omitted rather than guessed at.

measSpec — tag 176, DCL ^MS

measSpec — one per measure, keyed by measure number. 2011 tag 176; DCL tag ^MS.

```
// byte order: little-endian
struct MeasSpec {
    uint16 width;           // +0x0    measure width in EVPU
    uint16 key;             // +0x2    key signature; 0 means inherit
    uint16 beats;          // +0x4    time signature numerator
    uint16 divbeat;        // +0x6    EDU per beat; 1024 = quarter
    uint16 flags;          // +0xa    low byte holds the barline nibble
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	width	measure width in EVPU
	0x02–0x03	2	uint16	key	key signature; 0 means inherit
	0x04–0x05	2	uint16	beats	time signature numerator
	0x06–0x07	2	uint16	divbeat	EDU per beat; 1024 = quarter
	0x0a–0x0b	2	uint16	flags	low byte holds the barline nibble

Example bytes (12 shown), tinted to match the table above.

0000 58 02 03 00 04 00 00 04 00 00 02 00 X.....

A key of 0 means “inherit”, not C major. A .musx omits the element entirely rather than writing zero. Emitting key="0" would silently turn every inheriting measure into C major.

The barline nibble is the low byte of the u16 at +10, not the byte at +10. ETF stores others data as two-byte values, so on a big-endian file that byte is at +11. Reading +10 either way took the high byte from all 37 big-endian DCL documents.

frameSpec — tag 146, DCL ^FR

frameSpec — a run of entries forming one layer of one measure. 2011 tag 146; DCL tag ^FR.

```
// byte order: little-endian
struct FrameSpec {
    uint8[] header;         // +0x0    era-dependent lead-in
    uint32 startEntry;      // +0x6    first entry number in this frame
    uint32 endEntry;        // +0xa    last entry number, inclusive
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x05	6	uint8[]	header	era-dependent lead-in
	0x06–0x09	4	uint32	startEntry	first entry number in this frame
	0x0a–0x0d	4	uint32	endEntry	last entry number, inclusive

Example bytes (14 shown), tinted to match the table above.

0000        65     6c    e...l...

The base offset differs by era: 4 for 2001, 6 for 2005. The reader distinguishes them by incidence count — a three-incidence spec is the 2001 shape. Reading the wrong base yields entry numbers that look plausible and are wrong, which is the worst kind of error this format offers.

When a spec has more than one incidence, a `startTime` u32 sits at +0, in EDU from the start of the measure.

gfhold — tag 1044, DCL ^GF

gfhold (“frame hold”) — keyed by two cmpers, (staff, measure). 2011 tag 1044; DCL tag ^GF. This is the record that connects a place in the score to its music.

```
// byte order: little-endian
struct GfHold {
    uint16 clefID;           // +0x0    clef in force for this measure
    uint16 frame1;           // +0x6    layer 1 frame id; 0 = layer empty
    uint16 frame2;           // +0x8    layer 2 frame id
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	clefID	clef in force for this measure
	0x06–0x07	2	uint16	frame1	layer 1 frame id; 0 = layer empty
	0x08–0x09	2	uint16	frame2	layer 2 frame id

Example bytes (10 shown), tinted to match the table above.

0000  01   00 00 00 00 00 29   00  )...

A frame of **0** means the layer is empty and should be omitted, matching a .musx, where an absent slot reads as “this layer holds nothing”.

The format provides four layer slots. Only the first two are decoded here; no corpus document was found carrying a third or fourth.

staffSpec — tag 231, DCL ^IS

staffSpec — one per staff, keyed by staff number. 2011 tag 231; DCL tag ^IS. Only the transposition field is decoded.

```
// byte order: little-endian
struct StaffSpec {
    uint16  transposition;           // +0x14  written-to-sounding interval
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
	0x14–0x15	2 uint16	transposition	written-to-sounding interval

Example bytes (22 shown), tinted to match the table above.

```
0000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0010  00 00 00 00 f9 ff                                ....
```

The staff's *name* is not here — it lives in the text pool, which is why a .mus converts with positional part names unless that pool is resolved.

tupletDef — tag 1072

tupletDef — 2011 tag 1072. The example reads “3 quarters in the time of 2 quarters”.

```
// byte order: little-endian
struct TupletDef {
    uint16  symbolicNum;           // +0x0    printed count, the 3 of a triplet
    uint16  symbolicDur;           // +0x2    printed unit in EDU
    uint16  refNum;                // +0x4    count these replace
    uint16  refDur;                // +0x6    unit they replace, in EDU
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2 uint16	symbolicNum	printed count, the 3 of a triplet
	0x02–0x03	2 uint16	symbolicDur	printed unit in EDU
	0x04–0x05	2 uint16	refNum	count these replace
	0x06–0x07	2 uint16	refDur	unit they replace, in EDU

Example bytes (8 shown), tinted to match the table above.

```
0000  03 00 00 04 02 00 00 04                                . . . . .
```

The sounded duration of an entry inside a tuplet is its written duration scaled by $(\text{refNum} \times \text{refDur}) / (\text{symbolicNum} \times \text{symbolicDur})$.

staffGroup — tag 1057

staffGroup — the braces and brackets down the left edge. 2011 tag 1057.

```
// byte order: little-endian
struct StaffGroup {
    uint16 startInst;           // +0x0    first staff in the group
    uint16 endInst;             // +0x2    last staff, inclusive
    uint16 bracketId;           // +0xa    bracket shape
    uint8  barlineFlags;        // +0x15   bit 0: barlines cross the group
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	startInst	first staff in the group
	0x02–0x03	2	uint16	endInst	last staff, inclusive
	0x0a–0x0b	2	uint16	bracketId	bracket shape
	0x15	1	uint8	barlineFlags	bit 0: barlines cross the group

Example bytes (24 shown), tinted to match the table above.

```
0000  01 00 04 00 00 00 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00 00 00 00
0010  00 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

lyricVerse — tag 1108

lyricVerse — 2011 tag 1108. The payload is an array of 20-byte slots, one per entry that sings.

```
// byte order: little-endian
struct LyricVerseSlot {
    uint16 number;           // +0x0    verse number
    uint16 syll;             // +0x2    syllable index into the text pool
    uint16 wext;             // +0x8    non-zero: word extends with a m...
};
```

	OFFSET	SIZE	TYPE	FIELD	MEANING
	0x00–0x01	2	uint16	number	verse number
	0x02–0x03	2	uint16	syll	syllable index into the text pool
	0x08–0x09	2	uint16	wext	non-zero: word extends with a melisma

Example bytes (20 shown), tinted to match the table above.

```
0000  01 00 07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

articAssign — tag 1009

articAssign — 2011 tag 1009. Points at an *articDef* (tag 121), which carries the glyph.

```
// byte order: little-endian
struct ArticAssign {
    uint16 definition;        // +0x0    key of the articDef this uses
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
 0x00–0x01	2	uint16	definition	key of the articDef this uses

Example bytes (4 shown), tinted to match the table above.

0000  0c  00 00 00  ...

instUsed — tag 159

instUsed — 2011 tag 159. A 24-byte slot per incidence, giving the score's staff order. This, not the staff number, is what a part list should follow.

```
// byte order: little-endian
struct InstUsedSlot {
    uint16  staff;                // +0x0    staff number, one per incidence
    uint16  staff[1];            // +0x18   the next slot, 24 bytes on
};
```

OFFSET	SIZE	TYPE	FIELD	MEANING
 0x00–0x01	2	uint16	staff	staff number, one per incidence
 0x18–0x19	2	uint16	staff[1]	the next slot, 24 bytes on

Example bytes (48 shown), tinted to match the table above.

0000  01  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 
0010 00 00 00 00 00 00 00 00 00  02  00 00 00 00 00 00 00 00
0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Recognised but only partly decoded

These records are read and their keys resolved, but their payload layouts are not fully established. They are listed so an implementer knows they exist rather than meeting them as unexplained bytes.

RECORD	TAG	WHAT IT CARRIES
articDef	121	the glyph an articAssign refers to; character offsets 0 and 2
repeatBack	203	backward repeat barline; target measure
repeatEndingStart	204	first and second ending brackets
repeatPassList	206	which passes an ending applies to
clefOptions	109	the clef definition table, read as a strided array